

Tuya Agentic-Kit Documentation

2026-04-15

Contents

Tuya Agentic-kit 介绍	2
功能介绍	2
接入流程	3
Sdk 介绍	3
Demo 代码 Build	4
快速开始	5
依赖	5
编译	5
运行	6
项目结构	6
更新 IoT SDK	6
设备配网与激活	7
App 展示二维码，设备扫码配网 (scan_by_device_pair_demo)	7
App 侧操作流程	7
设备侧实现流程	11
关键 API	11
运行示例	12
注意事项	12
设备展示二维码，App 扫码配网 (scan_by_app_pair_demo)	13
整体流程	13
关键 API	13
设备侧实现要点	14
运行示例	14
与”设备扫码”方式的对比	15
注意事项	15
通过 OpenAPI 获取 Token 激活设备 (activate_demo)	15
整体流程	15
涉及的 OpenAPI	16
Token 格式	17
OpenAPI 签名机制	17

环境配置	18
运行示例	18
注意事项	20
语音/文本聊天 Demo	20
功能概述	20
运行方式	20
完整流程	21
代码详解	21
注意事项	23
拍学机（图片理解） Demo	23
功能概述	23
运行方式	23
完整流程	24
代码详解	24
与平台工作流的配合	26
图片要求	26
注意事项	27
配置定制 Agent	27
配置工作流	27
相应代码开发	29
STM Open SDK 使用文档	30
1. 概述	30
2. 前期准备	31
3. SDK 初始化与配置	31
4. 会话管理	34
5. AI 调用与数据传输	38
6. 错误处理	43
7. 快速开始	45
8. 附录	49
IoT Client API Reference	50
错误码	50
枚举类型	51
配置结构体	51
API 函数	52

Tuya Agentic-kit 介绍

功能介绍

Tuya Agentic-kit 是用于智能硬件接入 TuyaAI 平台的多模态端侧 sdk, 支持对话, 图 片理解/生成, 并且支持设备侧 MCP, 功能强大灵活, 对话低延迟, 可应用于各类 AI 硬 件场景.

Tuya AI 平台是全球化平台, 支持设备连接到全球不同的数据中心 (在配网时由配网用 户所在地决定), 默认支持设备的出海.

Tuya Agent-kit 是芯片和操作系统无关的, 既支持 mac/linux, 也支持 freertos, 既支持 x86 的, 也支持 mips, esp32 等芯片的. (当前只支持部份, 不同操作系统和芯片的适配正在 进行中).

接入流程

智能硬件要接入 Tuya 平台, 需要至少两类信息

- **产品 pid**, 这个上面配置一类设备的共同信息, 比如产品的功能点, 产品的面板, 以及产品的 ai agent 等, 这些都是配置在产品上的.
- **设备授权码**: 有了 pid 之后, 硬件还需要设备的授权码才可以连到云平台.

对于一个从头开始的开发者来说, 大致的流程是:

- 注册 Tuya IoT 平台账号, 创建产品, 并绑定或者创建 Agent
- 从 Tuya IoT 平台获取设备授权码信息 (uuid, authkey)
- 通过 App 配网操作, 激活授权码, 在激活时, tuyu 云端会生成 dev-id, sec-key, local-key 用于后续设备和云端交互.
- 通过 device-id, sec-key, local-key 来连接 AI 基座.

如果需要大规模出货, 需要联系 tuyu 的商务购买授权码. 客户可以在 Tuya lot 平台申 请免费的开发用授权码.

在接着往下读后续内容之前, 请最好读一下以下文档以了解相应的概念.

- [创建产品](#)
- [创建 Agent](#)

Sdk 介绍

当前 Agentic-kit 主要通过 libstm 库提供功能, 我们提供不同平台的预编译包. 当前可以支持.

- macos_arm64: 可以在 mac 上进行开发验证
- linux-gnu-amd64: linux 上 amd64 架构版本
- ingenic-mips: 君正的 MIPS 架构版本
- rockchip830-arm: linux 上的 rockchip 架构版本

以上所有平台预编译包可以在 libs 目录下找到. 另外我们正在适配 FreeRTOS 的 esp32 版本. 在 lib 目录下还有 libiot_sdk.

libstm 主要用于 AI 相关功能, libiot_sdk 主要用于会话令牌获取. 主要的使用流程如 下:

iot_sdk 使用

iot sdk 主要用于从 tuyu 云端获取会话令牌给 libstm 使用. 它的主要使用模式是:

```
iot_client_init(config)           // 使用 devid, secret_key, local_key, region, env 初始化
    |
    v
iot_client_get_session_token()    // 从涂鸦云端获取 AI 会话令牌
    |
    v
... 使用 token ...
    |
```

```

      v
iot_client_deinit()           // 清理资源

```

配置字段: `devid`, `secret_key`, `local_key`, `region` (如 CN) , `env` (如 PROD).

`region` 字段可支持配置不同的 tuya 数据中心, 具体值可参考头文件的枚举值以及平台上的文档. `region`, `devid`, `secret_key` 和 `local_key` 来自于 app 的配网流程, 可参考 [设备配网与激活](#) 部份文档的说明.

libstm 使用

libstm 主要用于 ai 功能的实现, 包括语音聊天, 图片识别, 图片生成等. 它的主要使用流程如下:

```

stm_open_init(config)           // 初始化 SDK, 设置日志回调
stm_open_set_log_level(level)   // 可选, 设置日志级别
      |
      v
stm_open_session_create(session_config) // 用 token + local_key 创建会话
      |                               // 注册 on_state 和 on_data_recv 回调
      v
(通过 on_state 回调等待连接状态变为 CONNECTED)
      |
      v
stm_open_session_send(session, data, fin) // 发送数据 (文本/音频/图片)
      |                               // fin=1 表示输入结束
      v
(通过 on_data_recv 回调接收响应: 文本、音频、命令)
(回调中 fin=1 表示响应结束)
      |
      v
stm_open_session_close(session)
stm_open_deinit()

```

具体更详细功能, 请参考 libstm 的[reference](#) 文档.

Demo 代码 Build

当前 demo 项目包含了两类 demo: `demo/ai/`下是聊天 (chat) 和拍学机 (`edu_camera`) 功能, `demo/pair/`以及 `demo/api-activate` 下是配网相关功能.

依赖安装和编译

项目依赖工具: `* cmake * make * python3` (用于执行云接口配网例子中的 Tuya OpenAPI 接口)

编译演示程序

```

cd agentik-demo
mkdir -p build && cd build
cmake .. && make

```

快速开始

依赖

工具/库	版本	macOS 安装	Linux (Debian/Ubuntu) 安装
CMake	≥ 3.22	brew install cmake	apt install cmake
libcurl	—	系统自带	apt install libcurl4-openssl-dev
MbedTLS 3.x	≥ 3.6	brew install mbedtls@3	apt install libmbedtls-dev

交叉编译环境无需预装 libcurl 和 MbedTLS，构建系统会自动从源码编译 (MbedTLS 3.6.5 + curl 8.12.1)。

编译

本机编译

```
mkdir -p build && cd build
cmake .. && make
```

CMakeLists.txt 会自动选择平台对应的预编译库目录：

平台	库目录
macOS arm64	libs/macos_arm64/
Linux x86_64	libs/linux-gnu-amd64/
Linux aarch64	libs/linux-gnu-aarch64/
Linux ARM (Rockchip830)	libs/rockchip830-arm/

其他可用预编译库（需手动选择）：libs/ingenic-mips/。

交叉编译 (Rockchip830)

使用交叉编译 Docker 镜像，镜像内已预置 arm-rockchip830-linux-uclibcgnueabi-hf-gcc 工具链：

```
# 启动编译容器 (x86_64 镜像)
docker run --platform linux/amd64 \
  -v $(pwd):/workspace/agentik-kit-demo \
  -it registry.shdocker.tuya-inc.top/gw/linux-amd64-build:v1.3.0 bash

# 容器内编译
cd /workspace/agentik-kit-demo
mkdir -p build_rockchip && cd build_rockchip
cmake -DCMAKE_TOOLCHAIN_FILE=../cmake/toolchain-rockchip830.cmake .. && make
```

交叉编译时 MbedTLS 和 libcurl 会自动从源码编译，产物为全静态链接，可直接拷贝到设备运行。

运行

纯文本对话

```
./build/chat_demo
```

语音对话 (16kHz / mono / 16-bit PCM)

```
./build/chat_demo input.pcm
```

拍学机 / 图片理解

```
./build/edu_camera_demo
```

设备配网 (二维码激活)

```
./build/pair_demo
```

设备凭证 (devid、secret_key、local_key) 默认硬编码在 src/chat_demo_main.c 和 src/edu_camera_demo_main.c 中, 可通过命令行参数覆盖。

项目结构

```
├── inc/                                # SDK 头文件
│   ├── iot_client.h                  # IoT 客户端 API
│   ├── stm_open.h                    # STM Open SDK API
│   ├── stm_typedef.h                 # 数据类型定义
│   └── stm_errno.h                   # 错误码定义
├── libs/                              # 各平台预编译静态库
├── src/
│   ├── chat_demo_main.c              # chat_demo 入口
│   ├── chat_demo.c                  # 语音/文本对话实现
│   ├── edu_camera_demo_main.c
│   ├── edu_camera_demo.c            # 图片理解实现
│   ├── pair_demo_main.c
│   └── pair_demo.c                  # 设备配网实现
├── doc/                              # 详细文档
├── cmake/
│   └── toolchain-rockchip830.cmake  # Rockchip830 交叉编译工具链
├── scripts/
│   └── update_iot_sdk.sh            # 更新 IoT SDK 脚本
```

更新 IoT SDK

从 GitLab Release 拉取最新 libiot_sdk.a 和头文件:

```
export GITLAB_TOKEN=<your-personal-access-token>
bash scripts/update_iot_sdk.sh
```

可选参数:

固定版本

```
bash scripts/update_iot_sdk.sh --tag v1.2.0
```

```
# 仅列出 assets, 不下载
bash scripts/update_iot_sdk.sh --list
```

脚本会自动将 libiot_sdk.a 放到对应的 libs/{platform}/ 目录, 并更新 inc/ 下的头文件。

GITLAB_TOKEN 需要 read_api 权限, 可在 https://registry.code.tuya-inc.top/-/profile/personal_access_tokens 创建。

设备配网与激活

新设备首次使用前, 需要通过配网操作将设备与涂鸦云的 app 账号绑定, 并激活授权码。当前 demo 展示了三种支持的配网方式:

	设备扫 App 二维码	设备展示二维码 App 扫	OpenAPI Token 激活
依赖涂鸦 App	是	是	否
设备硬件要求	摄像头	屏幕	无特殊要求
Token 来源	App 二维码中	云端 MQTT 推送	OpenAPI 返回
网络信息传递	二维码含 WiFi 凭据	设备需自行联网	设备需自行联网
适用场景	带摄像头的设备	带屏设备	自有 App / 产线激活 / 无屏无摄像头设备
对应 demo	scan_by_device_pair_demo	scan_by_app_pair_demo	activate_demo + tuya_openapi.py

激活成功后, 云端会为设备分配 devid、secret_key、local_key, 后续使用 `iot_client_init()` 以及 `stm_open_*` 接口时均需使用这三个字段。

App 展示二维码, 设备扫码配网 (scan_by_device_pair_demo)

本章介绍如何通过扫描涂鸦 App 生成的二维码, 完成设备的配网激活流程, 对应示例代码为 `demo/pair/scan_by_device_pair_demo.c`。

使用这种配网方式时, 需要设备带摄像头功能, 可以扫描 App 生成的二维码, 如果设备没有摄像头硬件, 则无法使用此配网方式。

涂鸦 App 在配网流程中会生成一个包含 Wi-Fi 凭据和激活 Token 的二维码, 设备端用摄像头扫描该二维码即可完成免手动输入的自动激活。

激活成功后, 云端会为设备分配 devid、secret_key、local_key, 后续使用 `iot_client_init()` 以及 `stm_open_*` 接口时均需使用这三个字段。

App 侧操作流程

第一步: 在涂鸦 App 首页点击右上角 “+”, 选择 “添加设备”。

第二步: 在设备类别列表中选择对应的产品品类 (例如: 摄像机 □ 智能摄像机 Wi-Fi)。

第三步: App 显示配网二维码, 提示用户将二维码对准设备摄像头, 保持 15-20cm 距离, 等待设备发出提示音。

二维码内容是一段固定格式的 JSON 字符串:



Figure 1: 添加设备入口

13:51

5G



添加设备



电工

摄像机

照明



智能摄像机
(Wi-Fi)



智能摄像机
(2.4GHz&5G
Hz)



智能摄像机
(蓝牙)

传感

大家电

小家电



云台机
(Wi-Fi)



智能球机
(Wi-Fi)



4G 摄像机
(4G)

厨房电器

运动健康



拍照门铃



智能门铃



智能门铃
(双频Wi-Fi)

摄像机/锁

网关中控



摄像头灯



视频基站



NVR

户外出行

节能能源



DVR



喂鸟器
(wifi)



移动摄像机

数码娱乐

智能门锁

工农业



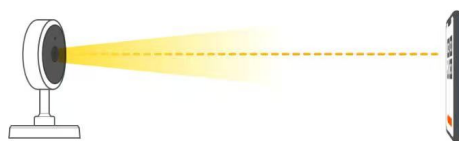
Figure 2: 选择产品品类

13:51

5G



将二维码正对摄像头，保持 15-20cm 距离



听到提示音

什么都没听到

Figure 3: 配网二维码

```
{"s":"<SSID>","p":"<password>","t":"<token>"}
```

其中:

字段	说明
s	Wi-Fi SSID
p	Wi-Fi 密码
t	涂鸦云激活 Token (前两字符编码 Region)

设备侧实现流程

scan_by_device_pair_demo 的完整流程如下:

```
stbi_load(jpg_path)           // 1. 解码 JPEG → 灰度像素缓冲
    |
    v
quirc_resize / quirc_begin / quirc_end // 2. quirc 定位并解码二维码
quirc_decode()
    |
    v
json_get_str("s") / ("p") / ("t") // 3. 解析 JSON, 提取 SSID / 密码 / Token
    |
    v
iot_client_init_on_boarding_with_token() // 4. 用 Token 激活设备, 获得 devid 等凭据
    |
    v
iot_client_get_session_token() // 5. 验证云端连通性, 获取 AI 会话令牌
    |
    v
iot_client_deinit()           // 6. 清理资源
```

关键 API

iot_client_init_on_boarding_with_token()

```
iot_client_t *iot_client_init_on_boarding_with_token(
    const iot_on_boarding_config_t *config,
    const char *token);
```

跳过 MQTT 等待, 直接使用已知 Token 发起激活请求。Region 由 Token 前两字符自动推导, 无需手动指定。

iot_on_boarding_config_t 主要字段:

字段	说明
uuid	设备 UUID (从涂鸦 IoT 平台申请的授权码)
authkey	设备 Auth Key
product_key	产品 PID
firmware_key	固件 Key (可为空)

字段	说明
timeout_ms	激活超时时间（毫秒）
env	环境：PROD / PRE
log_func	日志回调（可选）

返回值：成功返回 `iot_client_t *`，其中包含激活后的 `devid`、`secret_key`、`local_key`，后续可直接用于初始化 `iot_client_init()`；失败返回 `NULL`。

运行示例

使用默认参数（代码内置的测试授权码 + *qr.jpg*）

```
./build/scan_by_device_pair_demo
```

指定参数

```
./build/scan_by_device_pair_demo qr.jpg <uuid> <authkey> <product_key> <firmware_key>
```

成功输出示例：

```
=== pair-demo ===
```

```
QR image      : qr.jpg
UUID          : tuyaXXXXXXXXXXXX
Product key   : p891xbkosae0dgda
```

```
[pair_demo] Image loaded: qr.jpg (400x400)
[pair_demo] QR payload: {"s":"MyWiFi","p":"password123","t":"AY..."}
[pair_demo] SSID      : MyWiFi
[pair_demo] Password: password123
[pair_demo] Token     : AY...
[pair_demo] Starting on-boarding with token...
[pair_demo] Activation successful!
[pair_demo] devid      : <device_id>
[pair_demo] secret_key : <secret_key>
[pair_demo] local_key  : <local_key>
[pair_demo] Session token acquired (len=1234)
```

激活成功后，请将输出的 `devid`、`secret_key`、`local_key` 保存到设备持久化存储中，后续通过 `iot_client_init()` 直接使用，无需重复配网。

注意事项

- 本 demo 使用 `stb_image` 从文件加载图片，实际产品中替换为摄像头实时帧 即可，只需将像素数据写入 `quirc_begin()` 返回的缓冲区。
- Token 具有时效性，App 显示二维码后需在有效期内完成扫码激活。
- `firmware_key` 在大多数场景下可传空字符串。

设备展示二维码，App 扫码配网 (scan_by_app_pair_demo)

本章介绍另一种配网方式：设备端生成并展示二维码，用户使用涂鸦 App 扫码完成配网激活。对应示例代码为 demo/pair/scan_by_app_pair_demo.c。

在上一章（设备扫码配网）中，配网流程是 App 显示二维码、设备用摄像头扫码。但在部分产品形态中，设备可能没有摄像头（例如带屏音箱、智能面板），却有屏幕可以显示二维码。此时可以反过来——设备端向涂鸦云请求一个激活 URL，将其编码为二维码展示在屏幕（或终端）上，用户用涂鸦 App 扫描该二维码即可完成配网。

激活成功后，设备同样会获得 devid、secret_key、local_key，后续使用方式与扫码配网完全一致。

整体流程

```
iot_get_qrcode_info()           // 1. 向涂鸦云请求激活 URL
    |
    v
qrcodegen_encodeText()         // 2. 将 URL 编码为二维码
print_qr_terminal()           //    在终端/屏幕上显示
    |
    v
iot_client_init_on_boarding()   // 3. 等待 App 扫码，完成激活
    |
    v
iot_client_get_session_token() // 4. 验证云端连通性
    |
    v
iot_client_deinit()            // 5. 清理资源
```

关键 API

iot_get_qrcode_info()

```
int iot_get_qrcode_info(const iot_qrcode_request_t *request,
                        iot_qrcode_response_t *response);
```

向涂鸦云请求一个用于配网激活的 URL。设备将此 URL 编码为二维码展示给用户。

iot_qrcode_request_t 字段：

字段	说明
uuid	设备 UUID（从涂鸦 IoT 平台申请的授权码）
authkey	设备 Auth Key
app_id	App ID（可为空字符串）
type	二维码类型（通常为 1）
env	环境：PROD / PRE

返回值：OPRT_OK 表示成功，response->url 中包含激活 URL（调用方需 free）。

iot_client_init_on_boarding()

```
iot_client_t *iot_client_init_on_boarding(const iot_on_boarding_config_t *config);
```

阻塞等待用户通过 App 扫码完成激活。内部会通过 MQTT 监听激活事件，当 App 扫码并确认配网后自动完成设备激活。

与 `iot_client_init_on_boarding_with_token()` 的区别：

- `init_on_boarding()` — 不需要预知 Token，通过 MQTT 等待 App 扫码触发激活
- `init_on_boarding_with_token()` — 需要已知 Token（从二维码解析或 OpenAPI 获取），直接发起激活

设备侧实现要点

1. 获取激活 URL：调用 `iot_get_qrcode_info()` 获得一个 URL
2. 生成二维码：使用 `qrcodegen` 库将 URL 编码为二维码
3. 展示二维码：
 - 终端场景：用 Unicode 方块字符打印到终端（demo 中的实现方式）
 - 有屏设备：渲染到 LCD 屏幕上
4. 等待扫码：调用 `iot_client_init_on_boarding()` 阻塞等待
5. 激活完成：返回的 `iot_client_t` 包含 `devid`、`secret_key`、`local_key`

运行示例

```
# 二维码模式：设备展示二维码，等待 App 扫码
./build/scan_by_app_pair_demo
```

```
# Token 模式：跳过二维码展示，直接用 Token 激活（用于调试）
./build/scan_by_app_pair_demo <token>
```

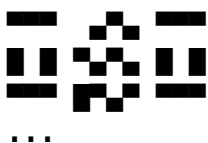
Token 格式为 {region}{activation_token}{secret}，其中 region 为 2 字符编码（如 AY 表示中国区）。

成功输出示例：

```
=== scan-by-app pair demo ===
UUID           : uuid9425d7b7f6060cbc
Product key    : 50fdsoli99bkgwri
Mode           : QR scan
```

```
=== QR Code URL ===
https://smartapp.tuya.com/...
```

```
=== Scan this QR code with the app to activate ===
```



```
=== On-boarding success ===
devid      : <device_id>
secret_key : <secret_key>
local_key  : <local_key>
```

```
=== Session Token ===
<session_token>
```

与”设备扫码”方式的对比

	设备扫码 (scan-by-device)	App 扫码 (本章)
二维码由谁生成	App 生成	设备生成
二维码由谁扫描	设备 (摄像头)	用户 (App)
设备硬件要求	需要摄像头	需要屏幕或终端输出
二维码内容	WiFi 凭据 + Token (JSON)	涂鸦云激活 URL
激活方式	init_on_boarding_with_token()	init_on_boarding()
网络信息传递	通过二维码传递 WiFi 信息	设备需自行联网

注意事项

- 此方式要求设备已具备网络连接能力 (WiFi 或以太网)，因为二维码中不包含 WiFi 凭据。
- `iot_client_init_on_boarding()` 会阻塞直到 App 扫码完成或超时 (`timeout_ms` 配置)，实际产品中建议在单独线程中调用。
- 本 demo 使用 `qrcodegen` (nayuki 库) 生成二维码，实际产品可替换为任意 QR 生成方案。

通过 OpenAPI 获取 Token 激活设备 (activate_demo)

本章介绍第三种配网方式：不依赖涂鸦 App，通过涂鸦 OpenAPI (Cloud API) 在 服务端完成用户创建和配网 Token 生成，再将 Token 传给设备完成激活。对应示例代码为 `demo/api-activate/`。

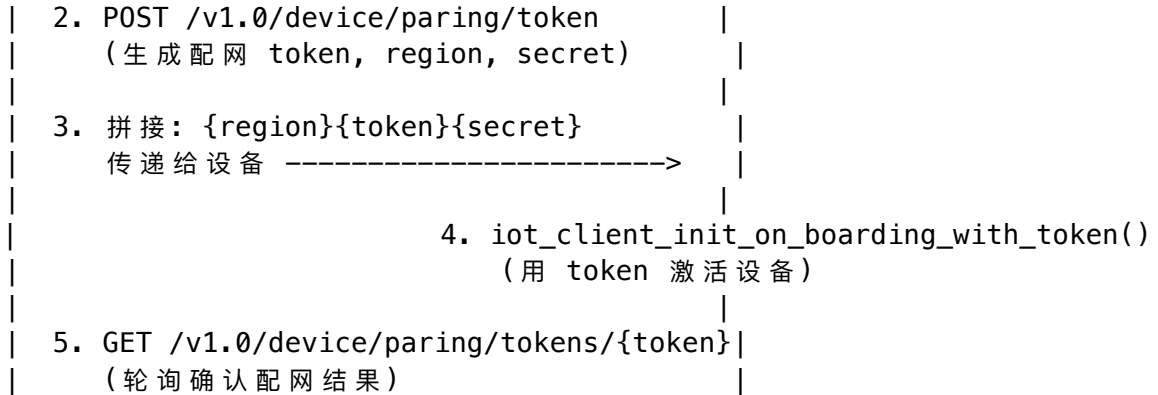
前两种配网方式都需要用户安装并使用涂鸦 App 来完成扫码配网。但在某些场景下，设备厂商可能：

- 拥有自己的 App 或后台系统，不希望依赖涂鸦 App
- 需要在生产线上批量激活设备
- 设备没有屏幕也没有摄像头

此时可以通过涂鸦 OpenAPI 直接在服务端完成用户同步和配网 Token 的生成，然后通过任意方式 (串口、蓝牙、HTTP 等) 将 Token 传递给设备，设备调用 `iot_client_init_on_boarding_with_token()` 即可完成激活。

整体流程

[服 务 端 / 脚 本]	[设 备 端]
1. POST /v1.0/apps/{schema}/user	
(创建/同步用户, 获得 uid)	



涉及的 OpenAPI

本方案用到两组涂鸦 Cloud API:

1. 用户同步—POST /v1.0/apps/{schema}/user

在涂鸦云中创建或同步一个用户，返回用户 uid，用于后续生成配网 Token。

参数	类型	必填	说明
country_code	string	是	国家码，如 "86"
username	string	是	用户名（手机号/邮箱/自定义）
password	string	是	密码（MD5 哈希值）
username_type	int	是	1= 手机号, 2= 邮箱, 3= 其他
nick_name	string	否	昵称
time_zone_id	string	否	时区，如 "Asia/Shanghai"

返回示例：

```
{
  "result": {
    "uid": "ay1776073832118bkJ5d"
  },
  "success": true
}
```

2. 生成配网 Token —POST /v1.0/device/paring/token

为指定用户生成设备配网 Token。

参数	类型	必填	说明
uid	string	是	用户 uid（来自上一步）
paring_type	string	是	配网类型：BLE、AP、EZ
time_zone_id	string	是	时区
home_id	string	否	家庭 ID

参数	类型	必填	说明
extension.uuid	string	否	设备 UUID (BLE 配网建议填写)

返回示例：

```
{
  "result": {
    "token": "H73H8u7A",
    "region": "AY",
    "secret": "p4pX",
    "expire_time": 100
  },
  "success": true
}
```

3. 查询配网结果—GET /v1.0/device/paring/tokens/{token}

查询设备是否已通过该 Token 完成激活。

返回示例：

```
{
  "result": {
    "success": [
      {
        "id": "6cdcabb1e507a8ea01u9pn",
        "name": "Smart Device",
        "product_id": "5gkeobhit9sd6odu"
      }
    ],
    "error": []
  },
  "success": true
}
```

Token 格式

设备端接收的 Token 需要由三部分拼接而成：

{region}{token}{secret}

例如：OpenAPI 返回 region="AY", token="H73H8u7A", secret="p4pX", 则传给设备的完整 Token 为 AYH73H8u7Ap4pX。

设备端的 `iot_client_init_on_boarding_with_token()` 会自动从前两个字符解析 Region 信息。

OpenAPI 签名机制

所有涂鸦 OpenAPI 请求都需要携带 HMAC-SHA256 签名。签名流程：

1. 获取 **Access Token**: GET /v1.0/token?grant_type=1, 签名源为 client_id + timestamp + stringToSign
2. 业务请求签名: 签名源为 client_id + access_token + timestamp + stringToSign, 其中 stringToSign = METHOD\nSHA256(body)\n\nurl_path

本 demo 提供的 tuya_openapi.py 脚本已完整实现签名逻辑, 可直接使用。

环境配置

使用前需设置以下环境变量:

环境变量	说明	示例
TUYA_CLIENT_ID	涂鸦 IoT 平台的 Access ID	xwm.....
TUYA_CLIENT_SECRET	涂鸦 IoT 平台的 Access Secret	95f13d4a616d407a...
TUYA_BASE_URL	OpenAPI 地址	https://openapi.tuyacn.com
SCHEMA	App schema 标识	marsid

Access ID 和 Access Secret 可在 [涂鸦 IoT 平台](#) 云开发 项目详情中获取。

不同数据中心对应的 TUYA_BASE_URL:

数据中心	URL
中国	https://openapi.tuyacn.com
美西	https://openapi.tuya.us.com
美东	https://openapi-ueaz.tuya.us.com
欧洲	https://openapi.tuya.eu.com
印度	https://openapi.tuya.in.com

运行示例

方式一: 使用一键脚本

activate-demo.sh 封装了完整的四步流程:

```
export TUYA_CLIENT_ID="your_client_id"
export TUYA_CLIENT_SECRET="your_secret"
export TUYA_BASE_URL="https://openapi.tuyacn.com"
export SCHEMA="your_schema"
```

```
./build/activate-demo.sh
```

脚本内部依次执行: 1. 调用 tuya_openapi.py sync-user 创建用户 2. 调用 tuya_openapi.py pairing-token 生成配网 Token 3. 调用 activate_demo 将 Token 传给设备完成激活 4. 调用 tuya_openapi.py pairing-result --poll 轮询确认结果

方式二：分步执行

1. 创建用户

```
python3 ./build/tuya_openapi.py sync-user \  
    --schema marsid --country-code 86 \  
    --username "test_user_001" --password "mypassword" \  
    --username-type 3 --time-zone-id "Asia/Shanghai"
```

2. 生成配网 Token

```
python3 ./build/tuya_openapi.py pairing-token \  
    --uid "ay1776073832118bkJ5d" --paring-type BLE \  
    --time-zone-id "Asia/Shanghai" --uuid "5682bceac872cfe7"
```

3. 拼接 Token 并传给设备激活

```
./build/activate_demo "AYH73H8u7Ap4pX" uuid17a65d2314ac60f5 \  
    tNn74X0lff222ocdUVVFYmjP15oWr9Vn 5gkeobhit9sd6odu
```

4. 确认配网结果

```
python3 ./build/tuya_openapi.py pairing-result --token "H73H8u7A" --poll
```

成功输出示例：

=== Step 1: Sync user ===

```
{  
  "result": { "uid": "ay1776073832118bkJ5d" },  
  "success": true  
}
```

=== Step 2: Generate pairing token ===

```
{  
  "result": { "token": "H73H8u7A", "region": "AY", "secret": "p4pX" },  
  "success": true  
}
```

Pairing token: AYH73H8u7Ap4pX

=== Step 3: Activate device ===

```
[api_activate] Activating with token: AYH73H8u7Ap4pX  
[api_activate] Activation successful!  
[api_activate] devid      : <device_id>  
[api_activate] secret_key : <secret_key>  
[api_activate] local_key  : <local_key>  
[api_activate] Session token acquired (len=1234)
```

=== Step 4: Poll pairing result ===

```
{  
  "result": {  
    "success": [{ "id": "<device_id>", "name": "Smart Device" }]  
  },  
  "success": true  
}
```

```
}
```

注意事项

- 配网 Token 有有效期（通常 100 秒），需在有效期内完成设备激活。
- `tuya_openapi.py` 使用 Python 标准库实现，无需安装额外依赖。
- `password` 参数传入原始密码，脚本会自动做 MD5 哈希后再发送给 OpenAPI。
- 实际产品中，OpenAPI 调用应在厂商自己的后台服务中完成，不应将 **Access Secret** 暴露在客户端或设备端。
- `username_type=3` 表示自定义用户名，适合厂商用内部用户体系同步。

语音/文本聊天 Demo

本章介绍 `chat_demo` 的功能与实现,对应源文件为 `demo/ai/chat_demo_main.c` 和 `demo/ai/chat_demo.c`。该 demo 演示了如何通过 Agentic-kit 实现与 AI 的语音或 文本聊天，并将 AI 返回的 TTS 音频保存到本地文件。

功能概述

`chat_demo` 支持两种模式：

- **语音聊天模式**：读取本地 16kHz/mono/16-bit 的 PCM 音频文件，按 120ms 帧分包上传，AI 会进行语音识别 (ASR) 并返回文本回复和 TTS 语音。
- **纯文本模式**：不提供 PCM 文件时，demo 会发送一段预设的文本问候语，AI 返回文本回复和 TTS 语音。

demo 代码要求的 input pcm 参数值：

参数	值
采样率	16000 Hz
声道数	1 (单声道)
位深	16-bit
格式	原始 PCM (无文件头)
帧时长	120 ms
帧大小	3840 字节

注：TuyaAI 云端支持 PCM/OPUS 音频，以及不同的采样率及其他参数，以 PCM 音频文件 仅为 demo 中简单起见写死的固定值，并非是必需的这样的格式。

两种模式都会：1. 在控制台实时打印 AI 返回的文本内容 2. 将 AI 返回的 TTS 音频保存到 `output_chat.pcm`
3. 统计并输出延迟信息（发送开始到首包文字、发送结束到首包音频等）

运行方式

```
# 纯文本模式（不需要 PCM 文件）
```

```
./build/chat_demo
```

```
# 语音聊天模式（需要 16kHz/mono/16-bit PCM 文件）
```

```
./build/chat_demo input.pcm
```

指定设备凭据

```
./build/chat_demo input.pcm <devid> <secret_key> <local_key>
```

完整流程

```
main()
├─ 解析命令行参数 (PCM 路径、设备凭据)
├─ iot_client_init(config)           // 初始化 IoT SDK
├─ iot_client_get_session_token()    // 获取 AI 会话令牌
├─ gen_session_id()                 // 生成 UUID 格式的 session_id
├─ demo_chat_run(token, local_key, session_id, input_pcm)
│   └─ 读取 PCM 文件到内存 (语音模式)
│   └─ stm_open_init()              // 初始化 STM SDK
│   └─ stm_open_set_log_level()      // 设置日志级别
│   └─ stm_open_session_create()     // 创建会话 (注册 on_state / on_data_recv 回调)
│   └─ 等待连接就绪 (on_state 回调收到 CONNECTED 状态)
│   └─ 发送数据
│       └─ 语音模式: send_audio_chunked() // 按 120ms 帧分包发送 PCM
│       └─ 文本模式: send_text()         // 发送文本, fin=1
│   └─ 等待 AI 响应 (on_data_recv 回调接收文本/音频/命令)
│   └─ 打印延迟统计
│   └─ stm_open_session_close() / stm_open_deinit()
└─ iot_client_deinit()
```

代码详解

入口 (chat_demo_main.c)

main() 负责初始化和令牌获取，核心业务逻辑封装在 demo_chat_run() 中。

// 1. 配置 iot_client

```
iot_client_config_t config = {
    .region = AY,           // 中国区
    .env     = PROD,        // 生产环境
    .log_func = iot_log_callback,
    ...
};
```

```
iot_client_t *client = iot_client_init(&config);
```

// 2. 获取会话令牌

```
iot_client_get_session_token(client, NULL, token);
```

// 3. 运行聊天 demo

```
demo_chat_run(token, local_key, session_id, input_pcm);
```

设备凭据 (devid、secret_key、local_key) 默认硬编码在代码中，也可通过命令行参数覆盖。这些凭据来自配网激活流程，具体可参考 [设备配网与激活](#) 章节。

会话创建与回调

创建会话时需要提供 session_token、session_id、encrypt_key (即 local_key)，以及两个核心回调函数：

```
stm_open_session_config_t sess_cfg = {0};
sess_cfg.session_token = token;
sess_cfg.session_id    = session_id;
sess_cfg.client_type   = STM_CLIENT_TYPE_DEVICE;
sess_cfg.encrypt_key   = local_key;
sess_cfg.on_state      = on_state;           // 连接状态变化回调
sess_cfg.on_data_recv  = on_data_recv;      // 数据接收回调
sess_cfg.user_data     = &ctx;             // 用户上下文

stm_open_session_t *session = stm_open_session_create(&sess_cfg);
```

on_state 回调：监听连接状态，当收到 STM_CONNECTION_STATE_CONNECTED 时 标记连接就绪。

on_data_recv 回调：处理 AI 返回的数据，按 data_type 分类处理：

data_type	处理方式
STM_DATA_TYPE_TEXT	打印到控制台，记录首包时间
STM_DATA_TYPE_AUDIO	写入 output_chat.pcm 文件，记录首包/末包时间
STM_DATA_TYPE_CMD	打印命令信息（如打断指令）

当回调参数 fin=1 时，表示本次 AI 响应完毕。

音频发送

语音模式下，PCM 数据按 120ms 帧（3840 字节）分包发送：

```
stm_open_data_t d = {0};
d.data_type          = STM_DATA_TYPE_AUDIO;
d.audio_params.codec_type   = 101;    // 原始 PCM
d.audio_params.sample_rate = 16000;
d.audio_params.channels    = 1;
d.audio_params.bit_depth   = 16;
d.audio_params.frame_duration = 120;  // 120ms
d.audio_params.frame_size   = 3840;   // 120ms 对应的字节数

// 循环分包发送
while (offset < pcm_len) {
    d.event_id = (offset == 0) ? event_id : NULL; // 仅首包携带 event_id
    d.payload  = pcm + offset;
    int8_t is_last = (offset + chunk >= pcm_len) ? 1 : 0;
    stm_open_session_send(session, &d, is_last); // 末包 fin=1
    offset += chunk;
}
```

要点: - event_id 仅在首包设置, 后续包置 NULL - 最后一个包的 fin 标志设为 1, 通知服务端数据发送完毕
- codec_type = 101 表示原始 PCM 格式

文本发送

纯文本模式下直接发送一条文本消息:

```
stm_open_data_t d = {0};  
d.data_type      = STM_DATA_TYPE_TEXT;  
d.event_id       = event_id;  
d.payload        = (uint8_t *)text;  
d.payload_length = strlen(text);  
stm_open_session_send(session, &d, 1); // fin=1, 一次性发送
```

注意事项

- 设备凭据需要通过配网流程获取, demo 中的默认凭据仅供测试使用。
- PCM 文件必须是无文件头的裸 PCM 数据, 不支持 WAV 等容器格式。
- AI 响应的最大等待时间为 60 秒, 超时后 demo 会退出。
- 连接建立的等待时间为 5 秒。

拍学机 (图片理解) Demo

本章介绍 edu_camera_demo 的功能与实现, 对应源文件为 demo/ai/edu_camera_demo_main.c 和 demo/ai/edu_camera_demo.c。该 demo 演示了如何通过 Agentic-kit 实现图片理解功能——发送一张图片和文本 prompt 给 AI, 接收结构化 JSON 结果和 TTS 语音回复。

功能概述

edu_camera_demo 模拟了 拍学机 的典型场景:

1. 用户按下拍照按钮, 设备拍摄一张照片
2. 设备将照片和触发指令 (prompt) 发送给云端 AI
3. AI 返回结构化 JSON (包含名称、拼音、英文等卡片信息) 和 TTS 语音播报

Demo 会: - 读取本地图片文件 (JPEG 或 PNG, 最大 10MB) - 先发送文本 prompt (fin=0), 再发送图片数据 (fin=1) - 在控制台实时打印 AI 返回的文本/JSON 内容 - 将 TTS 音频保存到本地文件, 并根据首包音频参数自动判断输出格式 (PCM/MP3/OGG 等) - 统计并输出延迟信息

运行方式

```
# 使用默认参数 (test.jpg + "image_recognition" prompt)  
./build/edu_camera_demo
```

```
# 指定图片和 prompt  
./build/edu_camera_demo my_photo.jpg "image_recognition"
```

```
# 完整参数  
./build/edu_camera_demo <img_path> <prompt> <audio_path> <devid> <secret_key> <local_ke
```

参数	默认值	说明
img_path	test.jpg	输入图片路径 (JPEG/PNG)
prompt	image_recognition	文本触发词, 对应平台工作流的分支
audio_path	output_tts.pcm	TTS 音频输出路径
devid	内置默认值	设备 ID
secret_key	内置默认值	设备密钥
local_key	内置默认值	本地密钥 (同时作为加密密钥)

完整流程

```
main()
├── 解析命令行参数 (图片路径、prompt、音频输出路径、设备凭据)
├── iot_client_init(config)           // 初始化 IoT SDK
├── iot_client_get_session_token()    // 获取 AI 会话令牌
├── gen_session_id()                 // 生成 UUID 格式的 session_id
├── demo_image_understand_run(token, local_key, session_id, img_path, prompt, audio_pa
    ├── 读取图片文件到内存, 自动检测格式 (JPEG/PNG)
    ├── stm_open_init()              // 初始化 STM SDK
    ├── stm_open_set_log_level()     // 设置日志级别
    ├── stm_open_session_create()    // 创建会话 (注册回调)
    ├── 等待连接就绪 (最多 10 秒)
    ├── 发送数据 (同一个 event_id)
    │   ├── send_text_prompt(event_id, prompt, fin=0) // 先发文本, fin=0
    │   └── send_image(event_id, img_data, fin=1)     // 再发图片, fin=1
    ├── 等待 AI 响应 (最多 60 秒)
    ├── 打印延迟统计
    └── stm_open_session_close() / stm_open_deinit()
└── iot_client_deinit()
```

代码详解

入口 (edu_camera_demo_main.c)

与 chat_demo 类似, main() 负责 IoT SDK 初始化和令牌获取, 核心逻辑封装在 demo_image_understand_run() 中。

图片格式检测

Demo 通过检查文件头魔数自动判断图片格式:

```
static uint8_t detect_image_format(const uint8_t *data, size_t len) {
    if (len >= 8 && data[0] == 0x89 && data[1] == 0x50 &&
        data[2] == 0x4E && data[3] == 0x47) {
        return 2; /* PNG */
    }
    return 1; /* 默认 JPEG */
}
```


对应 `stm_image_params_t.format` 的值: 1 = JPEG, 2 = PNG。

数据发送: 文本 + 图片

拍学机场景的关键在于同一个 **event** 中先发文本 **prompt** 再发图片, 使用相同的 `event_id`, 通过 `fin` 标志控制数据边界:

```
// 生成随机 event_id
char event_id[64];
generate_event_id(event_id, sizeof(event_id));

// 第一步: 发送文本 prompt, fin=0 (还有后续数据)
stm_open_data_t data = {0};
data.data_type      = STM_DATA_TYPE_TEXT;
data.event_id       = event_id;
data.payload        = (uint8_t *)prompt;
data.payload_length = strlen(prompt);
stm_open_session_send(session, &data, 0);    // fin=0

// 第二步: 发送图片, fin=1 (本次请求结束)
stm_open_data_t img = {0};
img.data_type      = STM_DATA_TYPE_IMAGE;
img.event_id       = event_id;
img.image_params.payload_type = 0;           // raw 数据
img.image_params.format      = format;      // 1=JPEG, 2=PNG
img.image_params.width       = 2048;
img.image_params.height      = 2730;
img.payload           = img_data;
img.payload_length     = img_len;
stm_open_session_send(session, &img, 1);    // fin=1
```

要点: - 文本 `prompt` 和图片属于同一个 `event` (相同 `event_id`) - 文本先发 (`fin=0`), 图片后发 (`fin=1`)
- `prompt` 内容 (如 "image_recognition") 需要和平台上 workflow 配置的选择器匹配

回调处理

`on_data_recv` 回调处理三种类型的下行数据:

data_type	处理方式
STM_DATA_TYPE_TEXT	打印 JSON/文本到控制台, 记录首包时间
STM_DATA_TYPE_AUDIO	首包时根据 <code>audio_params</code> 自动判断音频格式并创建对应后缀的输出文件, 将音频数据写入文件
STM_DATA_TYPE_CMD	打印命令信息

音频格式自动检测逻辑:

```
static const char *audio_output_ext(uint16_t codec_type, uint16_t container) {
    if (container == 1) return ".ogg";
```

```

switch (codec_type) {
case 109: return ".mp3";
case 101: return ".pcm";
case 108: return ".spx";
default: return ".pcm";
}
}

```

AI 响应示例

一次完整的图片理解请求会产生多个下行文本包：

```
[Text] {"bizId":"img_understand_001","bizType":"NLG","eof":0,"data":{"content":
  "{\\\"type\\\":\\\"card\\\",\\\"name\\\":\\\"谷歌浏览器图标\\\",\\\"pinyin\\\":\\\"gǔ gē liú lǎn qì tú biāo\\\",
  \\\"relatedWord\\\":\\\"Google Chrome Icon\\\"}"},"...}}
```

```
[Text] {"bizId":"img_understand_001","bizType":"NLG","eof":0,"data":{"content":
  "谷歌浏览器（Google Chrome）是谷歌公司开发的一款全球流行的网页浏览器..."},"...}}
```

```
[Text] {"bizId":"img_understand_001","bizType":"NLG","eof":1,"data":{"content":"","
  "finish":true,...}}
```

其中：- 第一个文本包包含 **结构化 JSON 卡片数据** (type: "card")，用于设备端 UI 显示 - 第二个文本包包含 **自然语言描述**，对应平台工作流中 TTS 输出分支的文本 - eof=1 且 finish=true 表示文本流结束

同时还会收到 TTS 音频包，保存到输出文件中。

与平台工作流的配合

edu_camera_demo 依赖在 Tuya AI 平台上配置的 **工作流 (Workflow)**。prompt 文本（如 "image_recognition"）在工作流中充当选择器的匹配条件，用于路由到 对应的处理分支。

工作流的典型结构：

```

开始 (USER_TEXT + USER_IMAGE)
├─ 选择器：根据 prompt 文本匹配分支
├─ 多模态大模型节点：图片识别，输出 JSON
├─ 卡片输出分支：将 JSON 直接输出给设备端
├─ TTS 输出分支：用大模型生成描述文本，再做 TTS
└─ 结束

```

关于工作流的详细配置方法，请参考 [配置定制 Agent](#)。

图片要求

参数	限制
格式	JPEG 或 PNG
大小	最大 10MB
传输方式	原始二进制 (payload_type=0)

注意事项

- prompt 文本必须和平台工作流中的选择器配置一致，否则可能无法触发正确的处理流程。
- 设备凭据需要通过配网流程获取，demo 中的默认凭据仅供测试使用。
- Demo 中图片的 width 和 height 参数是示意值，实际使用时应设置为真实图片的尺寸。
- AI 响应的最大等待时间为 60 秒，连接建立的最大等待时间为 10 秒。
- 输出音频文件的格式（后缀）会根据云端返回的首包音频参数自动判断。

配置定制 Agent

阅读此文前请先了解 Agent 配置以及工作流相关概念。可以阅读 [创建 Agent](#)

默认的 Agent 配置主要是用于语音聊天场景的，如果客户需要实现其他的功能场景，比如图片理解，或者以文生图等，需要在 Tuya 平台上以工作流方式配置 Agent 以实现。下面以配置一个 **拍学机** 的 Agent 为例，来说明如何配置工作流以实现功能。

一般 **拍学机** 的主要功能场景是：用户按一下拍照按钮，然后拍学机屏幕会显示关于 检测到的物体的相关信息，包括拼音/名称/英文/介绍等，这部份信息是 **结构化** 的，但字段可能每个拍学机的厂商是不同的。为实现此功能，我们需要创建一个工作流，然后输出一个 json，json 里有我们需要的字段。

配置工作流

当我们创建工作流的时候，画布上已经有了开始和结束节点，其中开始节点有 USER_TEXT 和 USER_IMAGE，其中 USER_TEXT 代表用户的文本输入，可以在 sdk 里以 text 类型的消息传，也可以是 audio 消息传上来之后云端 ASR 识别后的文本。

我们在这里加一个选择器，让他走到不同的流程（考虑到中英文等不同多语言场景）。

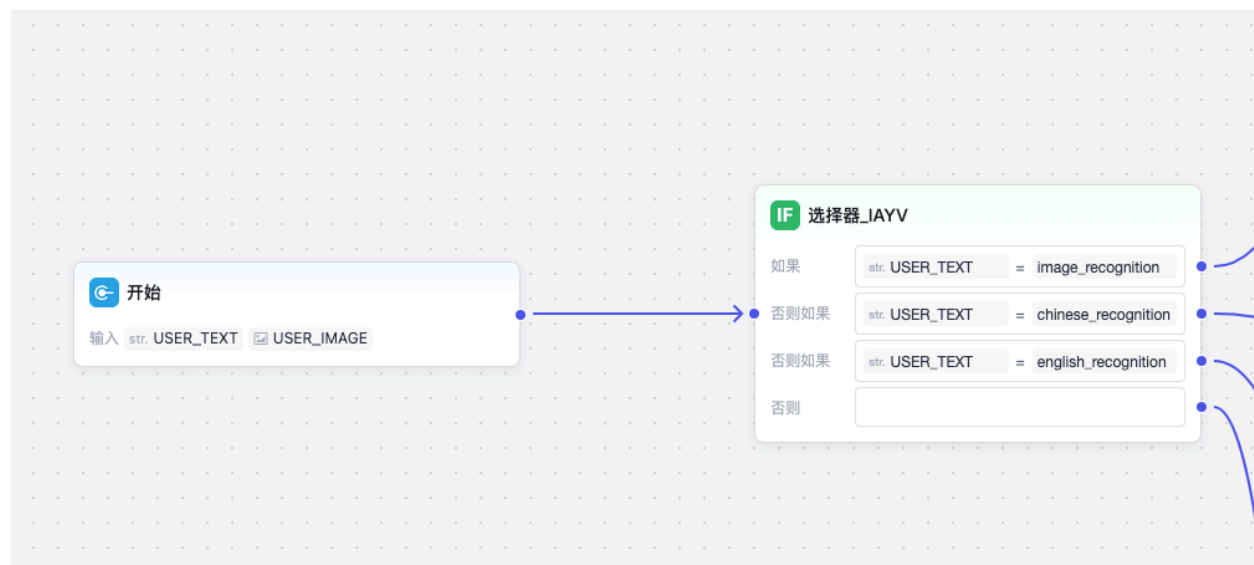


Figure 4: 工作流开始

然后我们配置一个多模态大模型节点，用于做图片识别。配置好提示词，输入（包括 文本和图片），以及输出，这里需要输出 json 格式。



Figure 5: 图片识别节点

大模型输出 json 后, 我们可以直接链接到输出结点, 完成整个流程. 我们也可以添加 额外的节点完成更多功能. 这里我们针对这个输出 完成两个功能

1. 词条的 json 输出, 在设备端做相应显示.
2. 再调用大模型生成对应 事件的”简介”, 然后完成 tts 输出, 用于完成设备端的语音播报

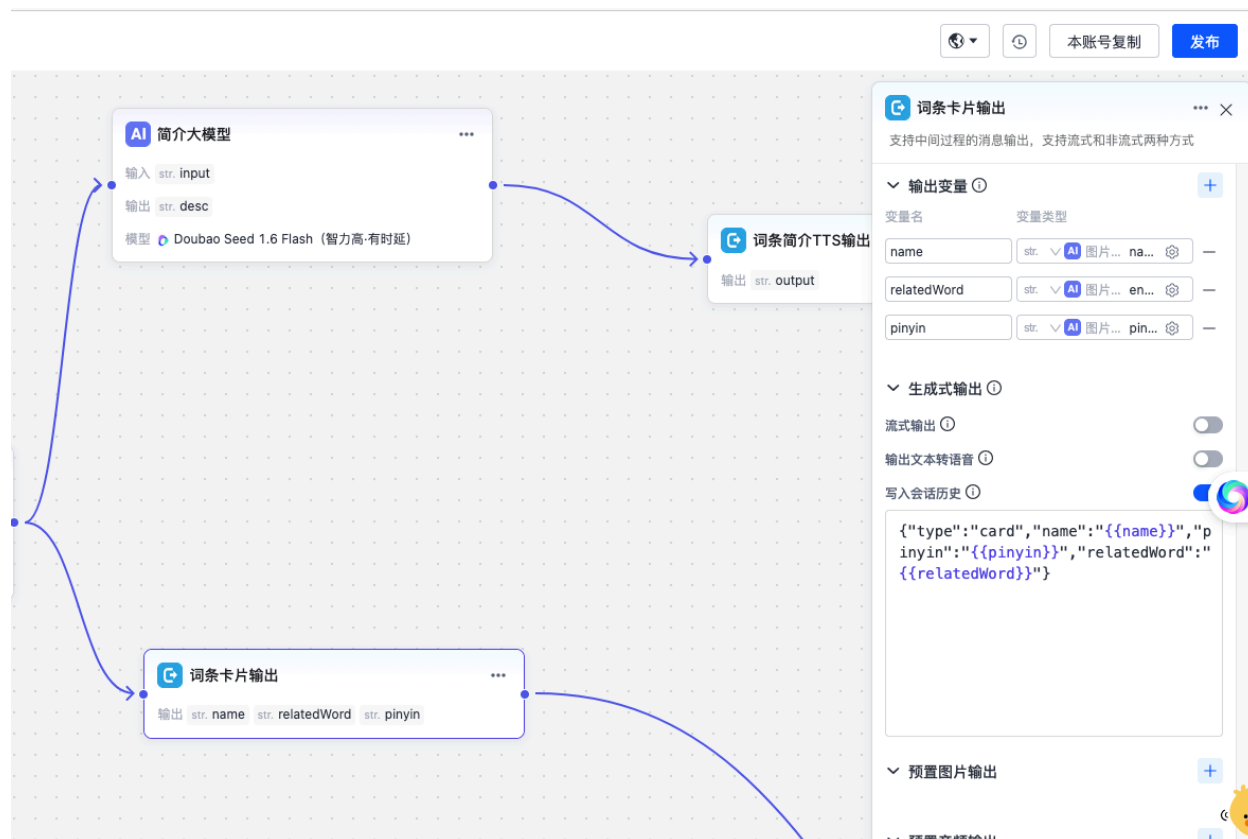


Figure 6: 卡片输出

最后把卡片输出和 tts 输出 连接到”结束”节点, 完成整个工作流的配置.

相应代码开发

```
//和工作流配置值对应的固定串
const char *prompt = "image_recognition";
...
//发送 prompt text, 用于触发工作流. `img_understand_001`是 bizId, 业务无关
stm_ret ret = send_text_prompt(session, "img_understand_001", prompt, 0);
...
//发送图片, fin 标志改为 1, 表示结束发包
ret = send_image(session, NULL, img_data, nread, img_format, 1);
...
//处理回调事件, 处理收到的 text 包里的 json
```

当程序跑出来后, 这个请求会产生两个输出: 一个 json 格式的结构化数据, 一个文本 串 (描述对应物体的).

数据发送完毕

等待 AI 响应...

```
[Text] {"bizId":"img_understand_001","bizType":"NLG","eof":0,"data":{"content":
  "{\"type\":\"card\",\"name\":\"谷歌浏览器图标\",\"pinyin\":\"gǔ gē liú lǎn qì tú biāo\",
  \"relatedWord\":\"Google Chrome Icon\"}"},"...}}
```

```
[Text] {"bizId":"img_understand_001","bizType":"NLG","eof":0,"data":{"content":
  "谷歌浏览器（Google Chrome）是谷歌公司开发的一款全球流行的网页浏览器..."},"...}}
```

```
[Text] {"bizId":"img_understand_001","bizType":"NLG","eof":1,"data":{"content":"","
  "finish":true,"...}}
```

STM Open SDK 使用文档

1. 概述

1.1 SDK 简介

STM Open SDK 面向外部开发者，在标准 STM SDK 的基础上做了流程简化，便于快速接入 AI 能力，无需理解 Connect、Stream、Event 等底层概念。

1.2 核心概念

1.2.1 Session（会话） 定义：

Session 表示 **AI 业务会话**，对应一个独立的对话或任务上下文。创建 Session 时需传入由服务端下发的 **session_token**；SDK 内部会据此完成连接与鉴权。

生命周期：

- 创建：调用 `stm_open_session_create(stm_open_session_config_t *config)`，传入 `client_type`、`token`、`id`、`encrypt_key` 及回调（如 `on_state`、`on_data_recv`），成功则返回 `stm_open_session_t*`。
- 使用：在同一 Session 上多次调用 `stm_open_session_send` 发送请求数据，在 `on_data_recv` 中接收 AI 返回。
- 关闭：业务结束或不再需要该会话时调用 `stm_open_session_close(session)` 释放资源。

举例：

用户打开一个「与 AI 对话」的页面 □ 应用调用平台接口获取 `session_token` 和 `session_id` □ 用 Open SDK 创建该 Session □ 用户发送文字或语音时通过 `stm_open_session_send` 发送，在 `on_data_recv` 里收到 AI 的回复并展示；用户离开页面时调用 `stm_open_session_close`。

1.2.2 请求与数据包（event_id、fin） 定义：

一次完整的「请求-响应」在数据上可能由**多包**组成（例如一段语音拆成多帧上传）。Open SDK 用**事件**来刻画这样一组包：同一轮请求/返回包共享同一个 **event_id**，**首包**可携带数据类型和参数（如音频采样率、图像宽高），**末包**通过参数 **fin=1** 标记，便于基座和端侧对齐「这一段输入已结束」。

发送侧（你调用 `stm_open_session_send` 时）：

- **event_id**: 事件 ID, 仅在本事件的第一包里填写 (后续包可置空或沿用同一 ID, 以协议约定为准), 用于基座和业务层关联同属一次请求的多个数据包。
- **data_type**: 数据类型 (如文本、音频、图像等), 首包时必填; 首包还可通过 union 填写对应的 **video_params / audio_params / image_params** 等, 描述编码、分辨率等。
- **payload / payload_length**: 本包的数据内容。
- **fin**: **0** 表示后面还有包, **1** 表示本包是该事件的最后一包, 基座收到 fin=1 后会对本次请求做统一处理。

接收侧 (**on_data_recv** 回调):

- 回调参数中的 **fin**: **0** 表示本轮 AI 返回还未结束, 后续还会有数据包到达; **1** 表示本条响应的最后一包, 可用于更新 UI (如“生成中”→“生成完成”)。
- 通过 **data** 中的 **data_type**、**payload** 等区分文本、音频、图片等, 按类型解码或处理, 通过 **event_id** 来关联本轮 AI 请求。

举例:

- 文本单轮对话: 发一包文本, fin=1; 收到多包或单包文本回复, 最后一包 fin=1。
- 语音多包上传: 第一包带 event_id、data_type= 音频、音频参数和第一段 payload、fin=0; 后续包只带 payload、fin=0; 最后一包 fin=1, 基座开始识别并返回结果。

2. 前期准备

在接入 Open SDK 前, 需在平台上完成智能体配置与设备激活, 并通过接口获取创建会话所需的 **session_token** 与 **session_id**。本节说明这些前置步骤, 便于开发者从零跑通一条完整链路。

2.1 智能体配置

(待补充: 描述如何在平台配置智能体。)

2.2 设备激活

(待补充: 描述设备的标准激活流程。)

2.3 获取 session token

(待补充: 调用 atop 等接口获取创建 Session 的配置信息, 重点包括 **session_token** 与 **session_id**, 用于在 4.1 节中调用 **stm_open_session_create**。)

3. SDK 初始化与配置

在使用会话与收发数据前, 必须先完成 Open SDK 的初始化与可选配置 (如日志回调、日志级别)。本节介绍初始化、重置、反初始化、版本查询与日志配置等 API。

3.1 SDK 初始化 (stm_open_init)

函数原型:

```
stm_ret stm_open_init(stm_open_config_t *config);
```

功能描述:

初始化 Open SDK, 设置日志回调等基础配置。该函数必须在调用其他 Open SDK API 之前调用, 且在整个应用生命周期内只能调用一次 (除非先调用 `stm_open_deinit` 进行卸载)。

参数说明:

参数名	类型	说明
config	stm_open_config_t *	Open SDK 配置结构体指针, 不能为 NULL

返回值:

返回值	说明
STM_OK	初始化成功
STM_EINVALID_PARM	参数无效 (config 为 NULL 或配置不合法)
其他负数	初始化失败, 参见 <code>stm_errno.h</code>

配置结构体 `stm_open_config_t`:

字段名	类型	说明
on_log	stm_log_cb_t	日志回调函数指针, 用于接收 SDK 输出的日志。可为 NULL, 表示不接收日志

3.2 SDK 重置 (stm_open_reset)

函数原型:

```
stm_ret stm_open_reset(stm_open_config_t *config);
```

功能描述:

将 Open SDK 重置到初始状态, 释放所有会话等资源, 并重新应用配置。调用后需重新创建 Session 才能继续使用; 若需完全卸载, 应使用 `stm_open_deinit`。

参数说明:

参数名	类型	说明
config	stm_open_config_t *	新的配置结构体指针，不能为 NULL，字段说明同 stm_open_init

返回值：

返回值	说明
STM_OK	重置成功
STM_ENOT_INIT	SDK 未初始化
STM_EINVALID_PARM	参数无效 (config 为 NULL)

3.3 SDK 反初始化 (stm_open_deinit)

函数原型：

```
void stm_open_deinit(void);
```

功能描述：

反初始化 Open SDK，释放占用的所有资源。调用后所有会话会被关闭，需要重新调用 stm_open_init 才能再次使用 SDK。

参数说明：

无。

返回值：

无 (void)。

注意事项： - 建议在调用前先关闭所有已创建的 Session (stm_open_session_close)。 - 反初始化后，再次调用除 stm_open_init 以外的 API 可能产生未定义行为或返回错误。

3.4 获取版本信息 (stm_open_get_version)

函数原型：

```
uint32_t stm_open_get_version(void);
```

功能描述：

获取 Open SDK 的版本号。版本号为 32 位整数，通常高 8 位为主版本、中 8 位为次版本、低 16 位为修订号。

参数说明：

无。

返回值：

返回值类型	说明
uint32_t	版本号, 例如 0x010000 表示 1.0.0

3.5 日志配置 (stm_open_set_log_level)

函数原型:

```
stm_ret stm_open_set_log_level(stm_log_level_e level);
```

功能描述:

设置 Open SDK 的日志输出级别。仅级别不低于所设级别的日志会通过 on_log 回调输出, 便于在开发时打开详细日志、在生产环境降低噪音。

参数说明:

参数名	类型	说明
level	stm_log_level_e	日志级别, 取值见下表

日志级别定义:

级别值	宏定义	说明
0	STM_LOG_LEVEL_VERBOSE	最详细
1	STM_LOG_LEVEL_DEBUG	调试
2	STM_LOG_LEVEL_INFO	信息 (推荐生产环境)
3	STM_LOG_LEVEL_WARN	警告
4	STM_LOG_LEVEL_ERROR	错误
5	STM_LOG_LEVEL_FATAL	致命错误
6	STM_LOG_LEVEL_NONE	不输出任何日志

返回值:

返回值	说明
STM_OK	设置成功
STM_ENOT_INIT	SDK 未初始化
STM_EINVALID_PARM	level 取值不合法

4. 会话管理

Open SDK 以 Session 为核心: 创建会话后即可在该会话上发送数据、接收 AI 返回。本节说明如何创建会话、发送数据、关闭会话, 以及会话状态与接收回调的含义。

4.1 创建会话 (stm_open_session_create)

函数原型:

```
stm_open_session_t* stm_open_session_create(stm_open_session_config_t *config);
```

功能描述:

使用平台下发的 session_token 与 session_id 创建业务会话。创建成功后返回会话句柄，用于后续发送数据与关闭会话；SDK 内部会据此完成连接建立与鉴权，开发者无需关心底层 Connect。

参数说明:

参数名	类型	说明
config	stm_open_session_config_t *	会话配置结构体指针，不能为 NULL

返回值:

返回值	说明
非 NULL	会话创建成功，返回 stm_open_session_t* 句柄，用于 stm_open_session_send 与 stm_open_session_close
NULL	创建失败（如未初始化、token 无效、网络异常等），可结合日志排查

配置结构体 stm_open_session_config_t:

字段名	类型	说明
client_type	stm_client_type_e	客户端类型，用于标识接入端的类型
session_token	char *	会话令牌，由平台/atop 等接口获取，不能为 NULL
session_id	char *	会话 ID，用于标识本会话，不能为 NULL。由平台分配或开发者自定义，建议在同一设备内保持唯一
encrypt_key	char *	加密密钥，用于会话数据加密，不能为 NULL
on_state	stm_open_session_on_state_change_t	会话状态变化回调，可为 NULL
on_data_rcv	stm_open_session_on_data_rcv_t	接收 AI 返回数据的回调，不能为 NULL
app_data	char *	应用自定义数据，会透传到各回调，可为 NULL

字段名	类型	说明
user_data	void *	用户自定义指针，会透传到各回调的 user_data 参数，可为 NULL

回调类型说明：

- 状态回调 stm_open_session_on_state_cb_t:
`void (*)(stm_open_session_t *session, uint16_t state, void *user_data);`
当会话状态变化（如连接成功、断开、错误等）时触发，state 取值由协议或业务约定，见 4.4 节。
- 接收回调 stm_open_session_on_data_recv_cb_t:
`void (*)(stm_open_session_t *session, stm_open_data_t *data, int8_t fin, void *user_data);`
当收到 AI 下发的数据时触发；data 为本次数据包，fin 表示该条响应是否结束（1= 最后一包），详见第 5 章。

4.2 发送数据 (stm_open_session_send)

函数原型：

stm_ret stm_open_session_send(stm_open_session_t *session, stm_open_data_t *data, int8_t fin);

功能描述：

在指定会话上发送一条上行数据包（如文本、音频、图像等）。同一请求若拆成多包发送，首包可带 event_id 与数据类型参数，末包需将 fin 置为 1，以便基座识别请求边界并开始处理。

参数说明：

参数名	类型	说明
session	stm_open_session_t *	由 stm_open_session_create 返回的会话句柄，不能为 NULL
data	stm_open_data_t *	本包数据 (event_id、data_type、params、payload 等)，不能为 NULL，字段说明见第 5.1 节
fin	int8_t	是否为本事件最后一包：0= 还有后续包，1= 最后一包，基座收到 fin=1 后会对整次请求做处理

返回值：

返回值	说明
STM_OK	发送成功（已写入发送队列或发出）
STM_ENOT_INIT	SDK 未初始化
STM_EINVALID_PARM	参数无效（session 或 data 为 NULL，或数据不合法）
其他负数	发送失败，参见 stm_errno.h

多包发送、首包带音频/图像参数等用法见第 5.1 节与第 7 章示例。

4.3 关闭会话 (stm_open_session_close)

函数原型：

```
void stm_open_session_close(stm_open_session_t *session);
```

功能描述：

关闭指定会话，释放与该会话相关的资源。关闭后不可再使用该句柄调用 stm_open_session_send；若需继续与 AI 交互，应重新调用 stm_open_session_create 创建新会话。

参数说明：

参数名	类型	说明
session	stm_open_session_t *	要关闭的会话句柄，不能为 NULL

返回值：

无 (void)。

注意事项： - 关闭后，该 session 指针不再有效，不应再被使用。- 建议在业务会话结束、页面退出或不再需要该会话时调用，避免资源泄漏。

4.4 会话状态说明

状态回调 on_state 的 state 参数：

state 为 uint16_t，具体取值由协议或业务约定。常见约定可包括（以实际协议为准）：

含义（示例）	说明
连接/会话就绪	可以开始发送数据
连接断开/异常	会话不可用，需关闭并可能重新创建
被服务端关闭	服务端主动结束会话

若协议文档中有明确的状态码表，请以协议为准；无约定时可在回调中打印 state 便于联调，或咨询平台侧。

接收回调 on_data_recv:

在 1.2.2 与第 5 章中已说明：通过 data->data_type 区分文本、音频、图像等，通过 fin 判断当前响应是否结束。回调最后一个参数为 user_data，即创建 Session 时传入的 user_data 指针。系统指令（如打断）处理见 5.2 节。

5. AI 调用与数据传输

本节用流程图和字段说明把「从初始化、创建 Session 到发送请求、接收 AI 返回」的完整数据流讲清楚，重点说明：上行如何组包（event_id、data_type、payload、fin）、下行如何根据 data_type 与指令类型（如 CMD 表示系统指令，如 break 表示聊天被打断）做处理。

5.0 整体数据流

从 SDK 初始化到多轮「发请求 □ 收响应」的完整流程如下（具体示例见第 7 章）：

sequenceDiagram

```
participant App as 应用
participant SDK as Open SDK
participant Server as AI 基座
```

```
App->>SDK: stm_open_init()
Note over App,SDK: 初始化（仅一次）
```

```
App->>SDK: stm_open_session_create(config)
SDK->>Server: 建连 / 鉴权（内部）
Server-->>SDK: 会话就绪
SDK-->>App: 返回 session 句柄
```

```
rect rgb(240, 248, 255)
Note over App,Server: 第 1 轮: 单包文本请求
App->>SDK: send(data{event_id="t1", TEXT, " 你好"}, fin=1)
SDK->>Server: 上行数据包
Server->>Server: AI 处理
Server->>SDK: 下行文本 (fin=0)
SDK->>App: on_data_recv(TEXT, " 你" fin=0)
Server->>SDK: 下行文本 (TEXT, " 好!" fin=1)
SDK->>App: on_data_recv(TEXT, fin=1)
Note over App: fin=1 → 第 1 轮响应结束
end
```

```
rect rgb(245, 255, 245)
Note over App,Server: 第 2 轮: 多包音频请求
App->>SDK: send(data{event_id="t2", AUDIO, params}, fin=0)
SDK->>Server: 首包 (带 audio_params)
App->>SDK: send(data{AUDIO, payload}, fin=0)
SDK->>Server: 中间包
App->>SDK: send(data{AUDIO, payload}, fin=1)
```

```

SDK->>Server: 末包 (fin=1)
Server->>Server: AI 处理
Server->>SDK: 下行音频 (fin=0)
SDK->>App: on_data_recv(AUDIO, fin=0)
Server->>SDK: 下行文本 (fin=0)
SDK->>App: on_data_recv(TEXT, fin=0)
Server->>SDK: 下行音频 (fin=1)
SDK->>App: on_data_recv(AUDIO, fin=1)
Note over App: fin=1 → 第 2 轮响应结束
end

rect rgb(255, 245, 245)
Note over App,Server: 第 3 轮: 请求被打断
App->>SDK: send(data{event_id="t3", AUDIO, payload}, fin=0)
App->>SDK: send(data{event_id="t3", AUDIO, payload}, fin=0)
SDK->>Server: 上行数据包
Server->>SDK: 下行数据包
SDK->>App: on_data_recv(event_id="t3", AUDIO, payload, fin=0)
Note over App: 用户中途说话触发打断
App->>SDK: send(data{event_id="t3", AUDIO, payload}, fin=0)
SDK->>Server: 上行数据包
Server->>SDK: 下行 CMD (break) 数据包
SDK->>App: on_data_recv(event_id="t3", CMD, fin=0)
Note over App: 收到 CMD → 停止播放、丢弃未完成内容
end

App->>SDK: stm_open_session_close(session)
App->>SDK: stm_open_deinit()

```

5.1 发送 AI 调用请求数据

上行数据通过 `stm_open_session_send(session, data, fin)` 发送，其中 `data` 为 `stm_open_data_t`。同一事件若拆成多包，**首包**需带 `event_id`、`data_type` 及对应参数 (union)，**末包**将 `fin` 置为 1。

数据结构 `stm_open_data_t`:

字段名	类型	说明
<code>event_id</code>	<code>char *</code>	事件 ID，由业务指定，保证 session 内唯一。 事件首包必填 ，后续包可置 NULL 或沿用同一 ID (以协议约定为准)
<code>data_type</code>	<code>stm_data_type_e</code>	数据类型，见下表。 事件首包必填 ，后续包可沿用或按协议填写

字段名	类型	说明
params*	union	事件首包必填。根据 data_type 填写对应成员： video_params / audio_params / image_params / file_params / text_params，描述编码、分辨率、采样率等，供基座正确解析 payload
payload	uint8_t *	本包负载数据（文本 UTF-8、音频帧、图像二进制等）。需指向有效内存；若本包无负载可为 NULL 且 payload_length 设为 0
payload_length	uint32_t	负载长度（字节）
app_data	char *	应用自定义透传数据，可选，可为 NULL

上行常用 data_type（发送时）：

值	宏	说明	union 成员
1	STM_DATA_TYPE_VIDEO	视频	video_params
2	STM_DATA_TYPE_AUDIO	音频	audio_params
3	STM_DATA_TYPE_IMAGE	图像	image_params
4	STM_DATA_TYPE_FILE	文件	file_params
5	STM_DATA_TYPE_TEXT	文本	text_params

参数结构体说明（首包按 data_type 填写对应 union 成员）：

以下 params 与标准 STM SDK 共用，定义在 stm_typedef.h 中。仅首包有效，后续包通常只需 payload / payload_length 与 fin。

1. video_params (stm_video_params_t) — 视频

字段名	类型	说明
codec_type	uint16_t	视频编码类型，常见值： 1-H.264, 2-H.265/HEVC, 3-VP8, 4-VP9, 5-AV1 等
sample_rate	uint32_t	视频采样率（Hz），通常用于时间戳计算，可设为帧率
width	uint16_t	视频宽度（像素），如 1920、1280、720
height	uint16_t	视频高度（像素），如 1080、720、480

字段名	类型	说明
fps	uint16_t	帧率 (帧/秒), 如 24、25、30、60
container	uint16_t	容器格式, 常见值: 1-MP4, 2-MKV, 3-AVI, 4-FLV 等
bitrate	uint32_t	码率 (bps), 如 2000000 (2Mbps)

2. audio_params (stm_audio_params_t) —音频

字段名	类型	说明
codec_type	uint16_t	音频编码类型, 常见值: 1-AAC, 2-MP3, 3-OPUS, 4-PCM, 5-G.711 等
sample_rate	uint32_t	采样率 (Hz), 如 8000、16000、44100、48000
channels	uint16_t	声道数, 1-单声道, 2-立体声
bit_depth	uint16_t	位深度 (比特/采样), 如 8、16、24、32
container	uint16_t	容器格式, 由业务约定
bitrate	uint32_t	码率 (bps)
frame_duration	uint16_t	帧时长 (ms)
frame_size	uint16_t	帧大小 (字节)

3. image_params (stm_image_params_t) —图像

字段名	类型	说明
payload_type	uint8_t	0-raw (二进制), 1-base64, 2-url
format	uint8_t	图像格式, 1-jpeg, 2-png
width	uint16_t	图像宽度 (像素), 可选
height	uint16_t	图像高度 (像素), 可选

4. file_params (stm_file_params_t) —文件

字段名	类型	说明
payload_type	uint8_t	0-raw, 1-base64, 2-url
format	uint8_t	文件格式, 1-mp4, 2-ogg, 3-pdf, 4-json, 5-log, 6-map 等
name	char[]	文件名, 最大长度 STM_FILE_NAME_MAX_LEN

5. text_params (stm_text_params_t) —文本 | 字段名 | 类型 | 说明 | |——|——|——|——|

文本类型通常无需额外参数，union 中 text_params 可为空结构体，payload 直接携带 UTF-8 文本即可。

发送约定小结： - 单包请求（如一句文本）：首包即末包，event_id、data_type、payload 填好，fin=1。
- 多包请求（如一段语音）：首包填 event_id、data_type、对应 params、首段 payload、fin=0；中间包填 payload、fin=0；末包填最后一段 payload、fin=1。

5.2 接收 AI 处理返回

下行数据通过创建 Session 时注册的 on_data_recv 回调推送，原型为：

```
void on_data_recv(stm_open_session_t *session, stm_open_data_t *data, int8_t fin, void
```

- **session**：当前会话句柄。
- **data**：本包数据，含 data_type、payload、payload_length 及首包时的参数 union。
- **fin**：0 表示该条 AI 响应还有后续包，1 表示本包为该条响应的最后一包，可用于更新 UI 状态（如“生成中”→“生成完成”）。
- **user_data**：创建 Session 时传入的 user_data 指针。

按 data_type 处理：

下行 data->data_type 使用与 stm_typedef.h 中一致的 stm_data_type_e 枚举，处理方式建议如下：

值	宏	说明	处理建议
0	STM_DATA_TYPE_CMD	系统指令	表示基座下发的控制类指令，非业务数据。当前常见为聊天被打断（如用户说话打断 AI 回复）。可根据 payload 或协议约定解析具体指令（如 break），停止播放 TTS、清空未完成内容等
1	STM_DATA_TYPE_VIDEO	视频	按协议解析 payload 为视频帧或 URL，首包可能有 video_params
2	STM_DATA_TYPE_AUDIO	音频	多为 TTS 音频，按 audio_params 与 payload 解码播放；可与文本混合到达
3	STM_DATA_TYPE_IMAGE	图像	文生图/理解结果等，按 image_params 与 payload 解码展示
4	STM_DATA_TYPE_FILE	文件	文件类结果，按 file_params 与 payload 处理
5	STM_DATA_TYPE_TEXT	文本	AI 回复文本，通常为 UTF-8，可直接展示或与音频配合

CMD 类型说明：

STM_DATA_TYPE_CMD 专用于系统指令，不表示业务内容。目前典型用法为打断 (break)：例如语音对话中用户中途说话，基座会下发 CMD 表示当前回复被打断，端侧应在收到后停止播放当前 TTS、丢弃后续未完成的该条响应包（直至收到 fin=1 或下一条新响应）。具体 CMD 子类型或 payload 格式以协议/平台约定为准。

6. 错误处理

接口通过返回值 `stm_ret (int32_t)` 表示成功或失败：**成功为 STM_OK (0)**，失败为负数错误码，定义在 `stm_errno.h`。本节列出 Open SDK 可能返回的各类错误码及**处理建议**。

6.1 返回值与错误码

成功：

返回值	宏	说明
0	STM_OK	执行成功

错误码列表与处理建议：

值	宏	含义	处理建议
-1	STM_ENOT_INIT	未初始化	确保在调用其他 API 前已调用 <code>stm_open_init</code> ；若已 <code>deinit</code> ，需重新 <code>init</code>
-2	STM_EINIT_MORE_THAN_ONCE	初始化超过一次	不要重复调用 <code>stm_open_init</code> ；若需重新配置，先 <code>stm_open_deinit</code> 再 <code>init</code> ，或使用 <code>stm_open_reset</code>
-3	STM_ETIMEOUT	超时	检查网络与基座可用性，适当重试；必要时提示用户检查网络
-4	STM_EINVALID_PARAM	无效入参	检查指针非 NULL、 <code>config/session/data</code> 字段合法（如 <code>token/id</code> 非空、 <code>payload_length</code> 与 <code>payload</code> 一致）
-5	STM_ENOT_CONNECTED	连接未建立	多为 <code>session</code> 尚未就绪；等待 <code>on_state</code> 就绪后再发数据，或检查 <code>token</code> 是否有效
-6	STM_ECONNECTION_CLOSED	会话已关闭	会话已不可用，调用 <code>stm_open_session_close</code> ，需重新获取 <code>token</code> 并创建新 <code>session</code>
-7	STM_EOUT_OF_CONNECTION	连接数已满	关闭不用的 <code>session</code> 后再创建新会话，或联系平台确认连接数限制
-8	STM_EMALLOC_FAILED	内存分配失败	检查设备内存与泄漏；可稍后重试或降级功能
-9	STM_EAUTH_FAILED	校验失败	检查 <code>session_token</code> 、设备鉴权信息是否正确；重新从平台获取 <code>token</code>
-10	STM_EHEARTBEAT_TIMEOUT	心跳超时	网络不稳定或基座无响应；提示用户检查网络，关闭当前 <code>session</code> 后重连
-11	STM_EINVALID_TOKEN	无效 TOKEN	<code>session_token</code> 无效或格式错误；重新调用平台接口获取有效 <code>token</code>
-12	STM_ETIME_OUT_NO_RESPONSE	超时未收到响应	基座未在约定时间内响应；可重试本次请求或提示用户

值	宏	含义	处理建议
-13	STM_ETIME_OUT_LOCAL	本地网络异常	检查本机网络、防火墙、DNS；提示用户检查网络设置
-14	STM_ETIME_OUT_REMOTE	远程网络异常	基座或中间网络异常；稍后重试或联系平台
-15	STM_ETOKEN_EXPIRED	Token 过期	重新获取 session_token 并创建新 session
-16	STM_ETOKEN_AUTH_FAILED	Token 认证失败	确认 token 未过期、未重复使用；重新获取 token
-17	STM_EINTERNAL_ERROR	内部错误	记录日志与复现步骤，升级 SDK 或联系支持
-18	STM_ENET_ERROR	网络错误	检查网络连通性；重试或提示用户
-19	STM_ENOT_SUPPORTED	不支持	当前接口或参数不被支持；查阅文档或升级 SDK
-20	STM_ENOT_FOUND	没有找到对象	如 session 已关闭仍被使用；确保使用有效的 session 句柄
-21	STM_EBUFFER_NOT_FULL	缓冲区满	单包 payload 过大或发送过快；减小单包大小（建议 ≤200KB）或控制发送节奏
-22	STM_ERECD_DATA_NOT_RECEIVED	接收数据不完整	多为网络丢包或中断；可等待后续包或按业务做超时/重试
-23	STM_ECMD_BUFFER_FULL	CMD 缓存满	发送过快；流控或稍后重试
-24	STM_ECREATE_STREAM_FAILED	创建流失败	SDK 内部建流失败；重试创建 session 或检查资源限制
-25	STM_ERECDV_NO_DATA	无接收数据	当前无数据可读；非致命，可按需重试
-26	STM_ERECDV_DATA_UNMATCHED	接收数据序列不匹配	协议或数据异常；记录 payload 便于排查，可丢弃本包
-27	STM_EAGAIN	需要重试（非阻塞）	操作暂未就绪；稍后重试同一操作
-28	STM_ETIMEDOUT	操作超时	单次操作超时；重试或加大超时时间（若可配置）
-29	STM_ESSL_HANDSHAKE_FAILED	SSL 握手失败	检查时间、证书、网络；确认基座地址与端口正确
-30	STM_ESSL_WRITE_FAILED	写入失败	连接可能已断开；关闭 session 后重连
-31	STM_ESSL_READ_FAILED	读取失败	同上，检查连接状态
-32	STM_ESSL_CONN_CLOSED	连接关闭	对端关闭连接；正常关闭 session，需新建 session
-33	STM_EENCRYPT_FAILED	加密失败	内部加密异常；重试或联系支持
-34	STM_EDECRYPT_FAILED	解密失败	解密异常（数据损坏或密钥不一致）；丢弃本包或重连
-35	STM_ESSL_CERT_PARSE_FAILED	证书解析失败	检查系统时间与证书链；联系平台获取正确证书或配置
-36	STM_ESSL_CERT_VERIFY_FAILED	证书验证失败	证书不受信或过期；确认证书配置与系统时间
-37	STM_ESSL_NO_PEER_CERT	无对端证书	服务端未提供证书；确认基座 TLS 配置
-38	STM_ESSL_CERT_INFO_READ_FAILED	读取证书信息失败	证书信息读取失败；检查证书有效性

使用建议：调用返回后若 ret != STM_OK，可根据上表打印或上报错误码，并执行对应处理（重试、重新获取

token、关闭 session、提示用户等)。

7. 快速开始

本节提供集成步骤与精简示例（文本聊天、音频聊天、文生图），关键处已加注释。完整流程见第 3、4、5 章。

7.1 集成 Open SDK

步骤 1：包含头文件

```
#include "stm_open.h" // 依赖 stm_typedef.h、stm_errno.h 等，需一并纳入编译
```

步骤 2：初始化与反初始化时机 - 在调用任何会话/发送 API 前调用一次 `stm_open_init(&config)`。
`config` 指针不能为 `NULL`，其中 `on_log` 字段可为 `NULL`（表示不接收日志）。- 应用退出或不再使用 AI 能力时调用 `stm_open_deinit()`；若需重新使用，需再次调用 `stm_open_init`。

7.2 文本聊天示例

单包文本上行、流式文本下行；上行首包即末包，`fin=1`。

接收回调：

```
void on_text_data_recv(stm_open_session_t *session, stm_open_data_t *data,
                      int8_t fin, void *user_data)
{
    if (data->data_type == STM_DATA_TYPE_TEXT) {
        // 将 payload 按 UTF-8 追加到界面文本区域
        // payload 可能分多包到达，每包拼接即可
        printf("%.s", data->payload_length, (char *)data->payload);

        if (fin == 1) {
            // 本轮 AI 回复结束，可更新 UI 状态（如 " 生成中" → " 生成完成"）
            printf("\n[回复结束]\n");
        }
    }
}
```

发送与主流程：

```
// 1. 初始化（仅一次）
stm_open_config_t config = {0};
config.on_log = my_log_cb;
if (stm_open_init(&config) != STM_OK) return -1;

// 2. 创建会话（token/id 来自平台）
stm_open_session_config_t sess_cfg = {0};
sess_cfg.client_type = STM_CLIENT_TYPE_DEVICE; // 按实际客户端类型填写
sess_cfg.session_token = "xxx"; // 从平台获取
```

```

sess_cfg.session_id    = "sid_001";
sess_cfg.encrypt_key   = "yyy"; // 从平台获取
sess_cfg.on_data_recv  = on_text_data_recv;
sess_cfg.user_data     = my_ctx; // 可选, 透传到回调
stm_open_session_t *sess = stm_open_session_create(&sess_cfg);
if (!sess) { /* 创建失败, 查 6.1 */ return -1; }

// 3. 发送单包文本 (fin=1 表示本条结束)
stm_open_data_t data = {0};
data.event_id = "t1";
data.data_type = STM_DATA_TYPE_TEXT;
data.payload = (uint8_t *) "你好";
data.payload_length = strlen("你好");
if (stm_open_session_send(sess, &data, 1) != STM_OK) { /* 处理错误 */ }

// 4. 等待 on_text_data_recv 回调接收 AI 回复...

// 5. 用完后关闭
stm_open_session_close(sess);
stm_open_deinit();

```

7.3 音频聊天示例

上行: 多包音频, 首包带 event_id、data_type、audio_params, 末包 fin=1。下行: 多包音频/文本混合, 需处理 STM_DATA_TYPE_CMD (打断)。

接收回调:

```

void on_audio_data_recv(stm_open_session_t *session, stm_open_data_t *data,
                        int8_t fin, void *user_data)
{
    switch (data->data_type) {
    case STM_DATA_TYPE_AUDIO:
        // 将音频 payload 写入播放缓冲区 (TTS 回放)
        // audio_player_write(data->payload, data->payload_length);
        if (fin == 1) {
            // 本轮 AI 音频回复结束
            // audio_player_flush();
        }
        break;

    case STM_DATA_TYPE_TEXT:
        // AI 返回的文本 (如字幕), 可与音频同步展示
        printf("%.s", data->payload_length, (char *)data->payload);
        break;

    case STM_DATA_TYPE_CMD:

```

```

        // 系统指令：当前典型为" 打断" (break)
        // 用户中途说话触发打断 → 停止播放当前 TTS、丢弃未完成响应
        // audio_player_stop();
        // ui_clear_pending_text();
        printf("[收到打断指令，停止播放]\n");
        break;

    default:
        break;
}
}

发送（上行多包音频）：

// 首包：填 event_id、data_type、audio_params、首段 payload、fin=0
stm_open_data_t first = {0};
first.event_id = "audio_event_1";
first.data_type = STM_DATA_TYPE_AUDIO;
first.audio_params = (stm_audio_params_t){
    .sample_rate = 16000,
    .codec_type = 3,      // OPUS
    .channels = 1,
    .bit_depth = 16
};
first.payload = pcm_buf;
first.payload_length = len;
stm_open_session_send(sess, &first, 0); // fin=0, 还有后续

// 中间包：只填 payload、fin=0
stm_open_data_t mid = {0};
mid.data_type = STM_DATA_TYPE_AUDIO;
mid.payload = pcm_buf2;
mid.payload_length = len2;
stm_open_session_send(sess, &mid, 0);

// 末包：fin=1, 本段语音结束，基座开始处理
stm_open_data_t last = {0};
last.data_type = STM_DATA_TYPE_AUDIO;
last.payload = pcm_buf3;
last.payload_length = len3;
stm_open_session_send(sess, &last, 1);

```

7.4 文生图示例

上行：发送文本描述。下行：IMAGE（AI 生成的图片）+ TEXT（说明文案）。

接收回调：

```

void on_image_data_recv(stm_open_session_t *session, stm_open_data_t *data,
                        int8_t fin, void *user_data)
{
    switch (data->data_type) {
    case STM_DATA_TYPE_IMAGE:
        // 收到 AI 生成的图片数据
        // 首包可通过 data->image_params 获取格式、宽高等信息
        // image_decoder_append(data->payload, data->payload_length);
        if (fin == 1) {
            // 图片数据接收完毕，解码并展示
            // image_show(decoded_image);
            printf("[图片接收完毕，展示图片]\n");
        }
        break;

    case STM_DATA_TYPE_TEXT:
        // AI 返回的文本说明 (如 " 这是一只橘猫")
        printf("%.s", data->payload_length, (char *)data->payload);
        if (fin == 1) {
            printf("\n[回复结束]\n");
        }
        break;

    default:
        break;
    }
}

```

发送 (上行文本描述):

```

// 单包文本请求, 描述想要生成的图片
stm_open_data_t text = {0};
text.event_id = "img_event_1";
text.data_type = STM_DATA_TYPE_TEXT;
text.payload = (uint8_t *) "画一只猫";
text.payload_length = strlen("画一只猫");
stm_open_session_send(sess, &text, 1); // fin=1, 请求结束

```

上行带参考图片 (可选, 若协议要求同一轮请求内附加图片):

```

// 第一包: 文本描述, fin=0 (后面还有图片包)
stm_open_data_t text = {0};
text.event_id = "img_event_2";
text.data_type = STM_DATA_TYPE_TEXT;
text.payload = (uint8_t *) "参考这张图, 画一只类似的猫";
text.payload_length = strlen("参考这张图, 画一只类似的猫");
stm_open_session_send(sess, &text, 0);

```

```

// 第二包: 参考图片, fin=1
stm_open_data_t img = {0};

```



```
img.data_type = STM_DATA_TYPE_IMAGE;
img.image_params = (stm_image_params_t){
    .payload_type = 0, // raw 二进制
    .format       = 1  // jpeg
};
img.payload = jpeg_buf;
img.payload_length = jpeg_len;
stm_open_session_send(sess, &img, 1); // fin=1, 请求结束
```

8. 附录

8.1 常量与类型定义

类型汇总 (Open SDK 相关):

类型名	说明
stm_open_config_t	初始化配置, 含 on_log
stm_open_session_t	会话句柄 (不透明), 由 stm_open_session_create 返回
stm_open_session_config_t	会话配置: client_type、session_token、 session_id、encrypt_key、on_state、 on_data_recv、app_data、user_data
stm_open_data_t	单包数据: event_id、data_type、 union(params)、payload、 payload_length、app_data
stm_open_session_on_state_cb_t	状态回调: (session, state, user_data)
stm_open_session_on_data_recv_cb_t	接收回调: (session, data, fin, user_data)

数据类型枚举 **stm_data_type_e**:
CMD(0)、VIDEO(1)、AUDIO(2)、IMAGE(3)、FILE(4)、TEXT(5), 见 5.1、5.2 节。
返回值类型: stm_ret (int32_t), 成功为 STM_OK (0), 失败见第 6 章。

API 快速参考:

API	返回值	说明	章节
stm_open_init(const stm_config_t)	stm_ret	初始化 SDK	3.1
stm_open_reset(const stm_config_t)	stm_ret	重置 SDK	3.2
stm_open_deinit(void)		反初始化 SDK	3.3
stm_open_get_version(void)	int32_t	获取版本号	3.4
stm_open_set_log_level(int level)	stm_ret	设置日志级别	3.5
stm_open_session_create(const stm_session_config_t*)	stm_open_session_t*	创建会话	4.1
stm_open_session_send(stm_open_session_t, data, fin)	stm_ret	发送数据	4.2

API	返回值	说明	章节
<code>stm_open_session</code> / <code>stm_close(session)</code>		关闭会话	4.3

8.2 常见问题 (FAQ)

Q1: Open SDK 与标准 STM SDK 如何选?

若为外部开发者、只需会话级收发、不想管 Connect/Stream/Event, 用 Open SDK; 若需多 Session 复用 Stream、精细控制 Event, 用标准 SDK。

Q2: session_token 从哪里获取?

通过 atop 等接口获取创建会话所需配置, 其中包含 session_token 与 session_id, 见第 2.3 节。

Q3: 同一 Session 内多轮事件如何关联请求与响应?

每轮请求事件使用不同 event_id; 接收下行数据时可在 on_data_recv 中与本地保存的 event_id 匹配, 实现一一对应。

Q4: 多会话并发要注意什么?

每个 session 独立句柄、独立回调; 注意连接数限制 (错误码 STM_EOUT_OF_CONNECTION), 不用时及时 stm_open_session_close。

Q5: 收到 CMD 类型数据怎么处理?

表示系统指令, 当前典型为“打断”; 应停止播放当前 TTS、丢弃该条未完成响应, 详见 5.2 节。

Q6: 发送失败返回 EBUFFER_NOT_ENOUGH 怎么办?

单包过大或发送过快; 减小单包 (建议 ≤200KB) 或做流控后重试。

8.3 版本历史

版本	说明
1.0.0	初始版本: Session 创建/发送/关闭, 上行 <code>stm_open_data_t</code> , 下行 <code>on_data_recv</code> , 支持文本/音频/图像/文件/CMD 等数据类型

IoT Client API Reference

libiot_sdk 提供设备激活、MQTT 连接和会话令牌获取功能。头文件: inc/iot_client.h。

错误码

值	宏	说明
0	OPRT_OK	执行成功
-1	OPRT_COMMUNICATION_ERROR	通信错误

值	宏	说明
-2	OPRT_INVALID_PARAMETER	无效参数
-3	OPRT_INVALID_RESULT	无效结果
-4	OPRT_UNINITIALIZED	未初始化
-5	OPRT_NOT_SUPPORTED	不支持
-6	OPRT_MALLOC_FAILED	内存分配失败
-7	OPRT_TLS_HANDSHAKE_FAILED	TLS 握手失败

枚举类型

Region (iot_region_t)

值	名称	说明
0	AY	中国
1	US	美西
2	UEAZ	美东
3	EU	欧洲
4	WEAZ	西欧
5	IN	印度
6	SG	新加坡

Environment (iot_env_t)

值	名称	说明
0	PROD	生产环境
1	PRE	预发布环境
2	TEST	测试环境

Log Level (log_level_t)

值	名称
0	LOG_LEVEL_ERROR
1	LOG_LEVEL_WARN
2	LOG_LEVEL_INFO
3	LOG_LEVEL_DEBUG

配置结构体

iot_client_config_t

用于已激活设备的初始化配置。

字段	类型	说明
devid	char[32]	设备 ID
secret_key	char[32]	设备密钥
local_key	char[32]	本地加密密钥
region	iot_region_t	数据中心区域
env	iot_env_t	环境
mqtt_disable_tls	bool	false (默认) 使用 MQTTS, true 使用明文 MQTT
log_func	log_func_t	日志回调函数, 可为 NULL
message_callback	iot_message_callback_t	MQTT 消息回调, 可为 NULL

iot_on_boarding_config_t

用于设备配网激活的配置。

字段	类型	说明
uuid	char[32]	设备 UUID (从涂鸦平台申请的授权码)
authkey	char[64]	Auth Key
product_key	char[32]	产品 PID
firmware_key	char[64]	固件 Key (可为空)
modules	const char *	模块信息 (可为 NULL)
feature	const char *	Feature 信息 (可为 NULL)
skill_param	const char *	Skill 参数 (可为 NULL)
timeout_ms	int	激活超时时间 (毫秒)
env	iot_env_t	环境: PROD (默认) 或 PRE
mqtt_disable_tls	bool	TLS 开关
log_func	log_func_t	日志回调
message_callback	iot_message_callback_t	MQTT 消息回调

iot_client_t (返回实例)

由 iot_client_init() 或配网 API 返回的客户端实例, 包含以下关键字段:

字段	类型	说明
devid	char[32]	激活后分配的设备 ID
secret_key	char[32]	MQTT 认证密钥
local_key	char[32]	本地加密密钥
region	iot_region_t	服务器区域
env	iot_env_t	环境

API 函数

iot_client_init

```
iot_client_t *iot_client_init(const iot_client_config_t *config);
```

使用已有设备凭据 (devid, secret_key, local_key) 初始化 IoT 客户端, 建立 MQTT 连接。

返回值: 成功返回 iot_client_t *; 失败返回 NULL。

iot_client_init_on_boarding

```
iot_client_t *iot_client_init_on_boarding(const iot_on_boarding_config_t *config);
```

阻塞等待 App 扫码激活。内部通过 MQTT 监听激活事件, 激活成功后返回包含 devid、secret_key、local_key 的客户端实例。

返回值: 成功返回 iot_client_t *; 超时或失败返回 NULL。

iot_client_init_on_boarding_with_token

```
iot_client_t *iot_client_init_on_boarding_with_token(  
    const iot_on_boarding_config_t *config,  
    const char *token);
```

使用预知的激活 Token 直接发起激活请求, 跳过 MQTT 等待。Region 由 token 前两个字符自动推导。

参数: - config — 配网配置 - token — 激活 Token (格式: {region}{token}{secret}, 如 AYH73H8u7Ap4pX)

返回值: 成功返回 iot_client_t *; 失败返回 NULL。

iot_client_deinit

```
void iot_client_deinit(iot_client_t *client);
```

反初始化 IoT 客户端, 断开 MQTT 连接, 释放所有资源。

iot_client_get_session_token

```
int iot_client_get_session_token(iot_client_t *client, const char *agent_code, char *to
```

从涂鸦云获取 AI 会话令牌 (session_token), 用于创建 STM Open SDK 会话。

参数: - client — IoT 客户端实例 - agent_code — Agent 代码 (传 NULL 使用默认 Agent) - token — 输出缓冲区, 接收 session token 字符串

返回值: OPRT_OK 成功; 其他为错误码。

iot_client_process

```
int iot_client_process(iot_client_t *client, uint32_t timeout_ms);
```

处理 MQTT 事件（接收消息、维持心跳）。在需要接收 MQTT 消息的场景下，应在循环中调用此函数。

参数：- client —IoT 客户端实例 - timeout_ms —处理超时时间（毫秒）

返回值：OPRT_OK 成功。

iot_client_publish

```
int iot_client_publish(iot_client_t *client, const uint8_t *data, size_t data_len);
```

向 smart/device/out/{deviceid} 发布加密消息。

参数：- client —IoT 客户端实例 - data —明文数据（内部自动加密） - data_len —数据长度

返回值：OPRT_OK 成功。

iot_get_qrcode_info

```
int iot_get_qrcode_info(const iot_qrcode_request_t *request,
                        iot_qrcode_response_t *response);
```

从涂鸦云获取配网激活 URL，设备可将此 URL 编码为二维码展示给用户。

请求参数 **iot_qrcode_request_t**：

字段	类型	说明
uuid	const char *	设备 UUID
authkey	const char *	Auth Key
app_id	const char *	App ID（可为空字符串）
type	int	二维码类型（通常为 1）
env	iot_env_t	环境

响应 **iot_qrcode_response_t**：

字段	类型	说明
url	char *	激活 URL（调用方需 free）

返回值：OPRT_OK 成功。

iot_get_ca_certificate

```
int iot_get_ca_certificate(iot_client_t *client, const char *host,  
                           uint16_t port, char **ca_certificate);
```

获取目标主机的 CA 证书。

参数：- client —IoT 客户端实例(可为 NULL)- host —目标主机名 - port —目标端口 - ca_certificate
—输出 CA 证书字符串（调用方需 free）

返回值：OPRT_OK 成功。

iot_set_log_callback

```
void iot_set_log_callback(log_func_t func);
```

设置全局日志回调。传 NULL 禁用日志输出。